

AI & Accessibility: NOPE!

(well, maybe)

Ashleigh Lodge

Leader, Enterprise Apps
(Dev, UX, QA)

Neovation Learning Solutions

[@ashleighlodge.ca](#)

[Linktree](#)

[ashleighlodge.ca](#)



Neovation



SmarterU



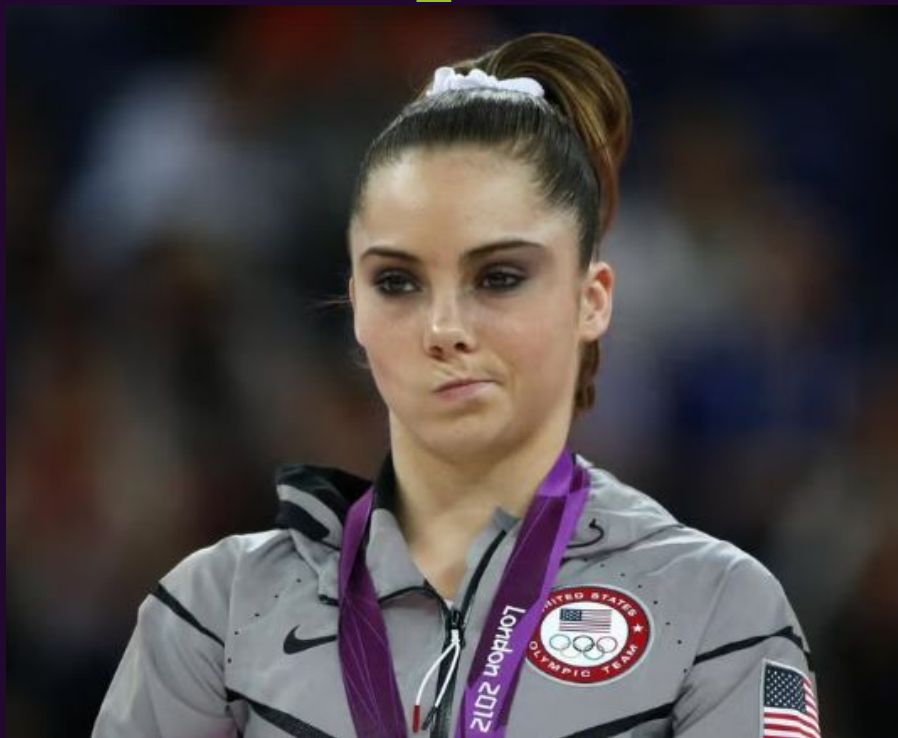
Neovation
Custom Learning



JoySuite

AI-First Everything

Not Impressed



Accessibility

===

Life

The Good, the Bad, the Ugly

The Experiment

The Experiment

One app,
5 versions

The Experiment

- New, never used Claude account
- Separate sessions & branches
- No system instructions, tools/skills, MCP servers initially
- Provide goals, not specific work
- Automated + manual accessibility reviews

The Experiment: Sites

Main branch



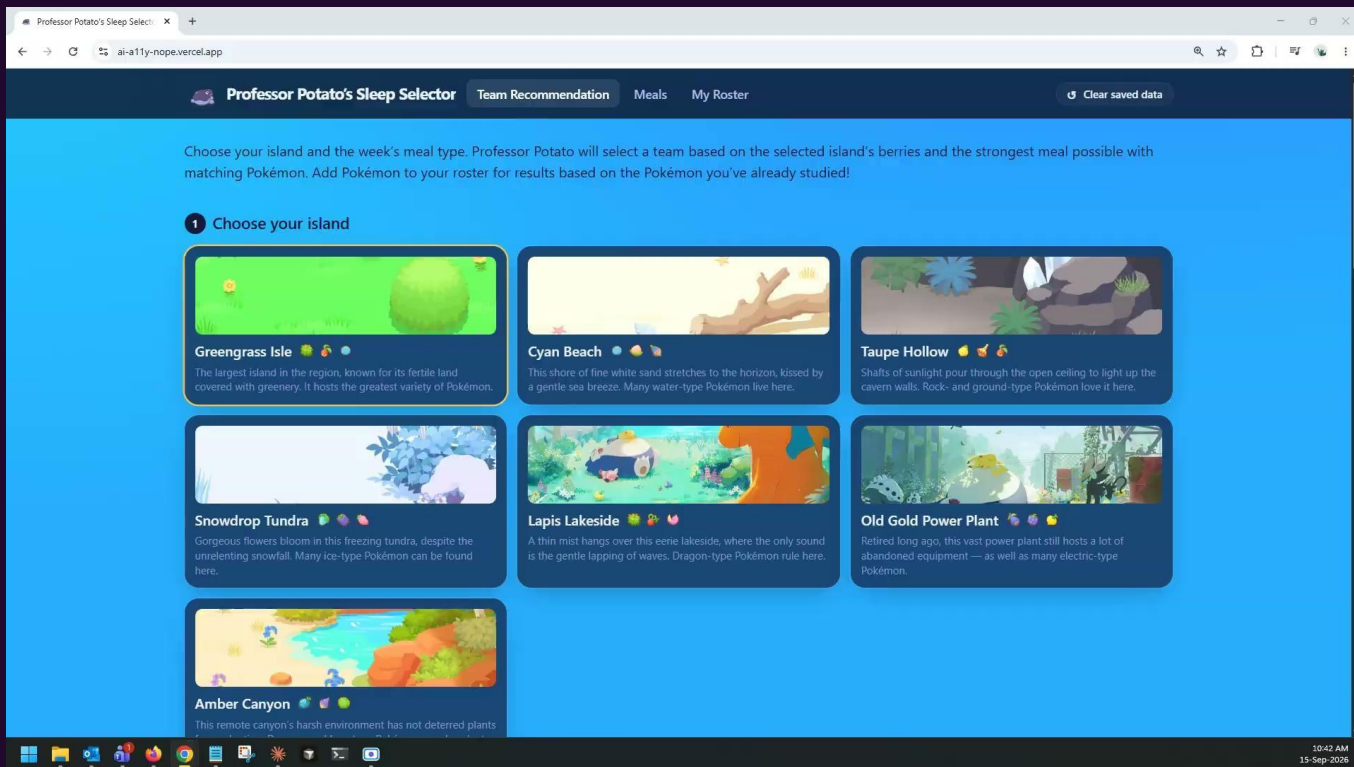
<https://ai-a11y-nope.vercel.app>

Version branches



<https://ai-a11y-nope-git-prdc-v1-rhoynar.vercel.app>

The Experiment: Home



The Experiment: Meals

 **Professor Potato's Sleep Selector** Team Recommendation **Meals** My Roster Clear saved data

Meals

Meals and ingredients are listed by their total strength, with the strongest at the top of each section. Meal strength is calculated by multiplying the number of each ingredient required to make the meal by that ingredient's strength, and doesn't take into account meal levels or bonuses. The default meal for each type is marked with a 🌟.

Meals 76 dishes

 **CURRIES & STEWS** 23

 Overgrow Avocado Gratin	115 ing.	13,934
 Cut Sukiyaki Curry	101 ing.	12,829
 Hidden Power Perk-Up Stew	92 ing.	11,839
 Role Play Pumpkaboo Stew	69 ing.	10,555
 Inferno Corn Keema Curry	77 ing.	9,250
 Dream Eater Butter Curry	55 ing.	6,674
 Ninja Curry	50 ing.	6,382
 Grilled Tail Curry	33 ing.	5,986
 Spicy Leek Curry	32 ing.	4,720 🌟
 Dizzy Punch Spicy Curry	33 ing.	4,224
 Limber Corn Stew	30 ing.	3,736

Ingredients 19 total

 Slowpoke Tail	342
 Plump Pumpkin	250
 Large Leek	185
 Tasty Mushroom	167
 Glossy Avocado	162
 Rousing Coffee	153
 Soothing Cacao	151
 Greengrass Corn	140
 Fiery Herb	130
 Soft Potato	124
 Pure Oil	121
 Fancy Egg	115

The Experiment: Roster

 **Professor Potato's Sleep Selector** Team Recommendation Meals **My Roster** [Clear saved data](#)

My Roster

Select the Pokémon you've studied — 3 selected. Roster data is saved to local storage and is only available in this browser, on this device. Roster data will be removed when browser data is cleared, or when using the Clear saved data option on the site.

[Clear selection](#)

 Bulbasaur ✓ Durin	 Ivysaur Durin	 Venusaur Durin	 Charmander Leppa
 Charmeleon Leppa	 Charizard Leppa	 Squirtle Oran	 Wartortle Oran
 Blastoise Oran	 Caterpie ✓ Lum	 Metapod Lum	 Butterfree Lum
 Rattata Persim	 Raticate Persim	 Ekans Chesto	 Arbok Chesto
 Pikachu Grepa	 Raichu Grepa	 Sandshrew Figy	 Sandslash Figy
 Clefairy Pecha	 Clefable Pecha	 Alolan Vulpix ✓ Rawst	 Vulpix Leppa

The Experiment: Main

Professor Potato's Sleep Selector

Team Recommendation

Meals

My Roster

Clear saved data

Choose your island and the week's meal type. Professor Potato will select a team based on the selected island's berries and the strongest meal possible with matching Pokémon. Add Pokémon to your roster for results based on the Pokémon you've already studied!

1 Choose your island



Greengrass Isle 🍌 🍍 🍓

The largest island in the region, known for its fertile land covered with greenery. It hosts the greatest variety of Pokémon.



Cyan Beach 🍌 🍍 🍓

This shore of fine white sand stretches to the horizon, kissed by a gentle sea breeze. Many water-type Pokémon live here.



Taupe Hollow 🍌 🍍 🍓

Shafts of sunlight pour through the open ceiling to light up the cavern walls. Rock- and ground-type Pokémon love it here.



Snowdrop Tundra 🍌 🍍 🍓

Gorgeous flowers bloom in this freezing tundra, despite the unrelenting snowfall. Many ice-type Pokémon can be found here.



Lapis Lakeside 🍌 🍍 🍓

A thin mist hangs over this eerie lakeside, where the only sound is the gentle lapping of waves. Dragon-type Pokémon rule here.



Old Gold Power Plant 🍌 🍍 🍓

Retired long ago, this vast power plant still hosts a lot of abandoned equipment — as well as many electric-type Pokémon.



Amber Canyon 🍌 🍍 🍓

This remote canyon's harsh environment has not deterred plants from adapting. Dragon- and bug-type Pokémon are abundant.

2 Choose your meal type



Curries & Stews

Simmered slow with roux, broth, and tender vegetables, these hearty bowls warm you from the inside. Rich, filling, and endlessly comforting.



Salads

Crisp vegetables and bright dressings, tossed fresh and light. From cool tofu to robust bowls, every forkful stays clean and lively.



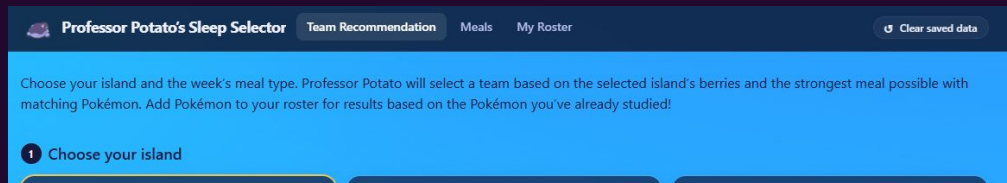
Desserts & Drinks

Sweet treats and refreshing sips — cakes, flans, smoothies, and teas. Some lull you toward sleep, others jolt you awake.

The Experiment: Main

- **axe: 5 issues**
- **WAVE: 8.4/10 lowest score**
- **Lighthouse: 96 lowest score**
- **Keyboard & screen reader: terrible**
- **Uncaught issues: many**

The Experiment: v1



I'm not an accessibility expert, but i know that contrast and alt text is important. Make sure that the contrast is good and that all the images have alt text



The Experiment: v1

- axe: **5 issues**
- WAVE: **8.4/10 lowest score**
- Lighthouse: **96 lowest score**
- Keyboard & screen reader: **terrible**
- Uncaught issues: **many**

The Experiment: v1

- 6 contrast fixes covering 10 areas
- Alt text:



The Experiment: v1

Alt text. Good news — every image already has an `alt` attribute, and they're used correctly. Worth explaining since you're not an a11y expert: there are two right ways to do alt text.

- **Informative images get descriptive alt:** Pokémon sprites (name), island art (island name), recommended dishes (dish name), island favorite-berry icons (berry name). 
- **Decorative images get *empty* `alt=""`:** the type/berry/ingredient chip icons, the nav logo, the pot icon, the “Professor Potato” mascot. Each sits right next to a text label saying the same thing, so an *empty* alt is the correct choice — it tells screen readers to skip the icon instead of announcing the same word twice. An empty `alt=""` is intentional and correct, not a missing alt.

So no alt changes were needed; the accessibility win here was entirely the contrast pass.

The Experiment: v1



Amber Canyon

This remote canyon's harsh environment has not deterred plants from adapting. Dragon- and bug-type Pokémon are abundant.



Desserts & Drinks

Sweet treats and refreshing sips — cakes, flans, smoothies, and teas. Some lull you toward sleep, others jolt you awake.



Clodsire

Ingredient

Chesto

Lv1

2 Cacao

Lv30

5 Cacao

or

4 Coffee

Lv60

7 Cacao

or

7 Coffee

or

9 Potato

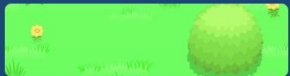
Clodsire receives Amber Canyon's berry bonus, and may provide Soothing Cacao for Sweet Scent Chocolate Cake.

The Experiment: v2

 **Professor Potato's Sleep Selector** [Team Recommendation](#) [Meals](#) [My Roster](#) [Clear saved data](#)

Choose your island and the week's meal type. Professor Potato will select a team based on the selected island's berries and the strongest meal possible with matching Pokémon. Add Pokémon to your roster for results based on the Pokémon you've already studied!

1 Choose your island



Greengrass Isle 🍌 🍍 🍓

The largest island in the region, known for its fertile land covered with greenery. It hosts the greatest variety of Pokémon.



Cyan Beach 🍌 🍍 🍓

This shore of fine white sand stretches to the horizon, kissed by a gentle sea breeze. Many water-type Pokémon live here.



Taupe Hollow 🍌 🍍 🍓

Shafts of sunlight pour through the open ceiling to light up the cavern walls. Rock- and ground-type Pokémon love it here.



Snowdrop Tundra 🍌 🍍 🍓

Gorgeous flowers bloom in this freezing tundra, despite the unrelenting snowfall. Many ice-type Pokémon can be found here.



Lapis Lakeside 🍌 🍍 🍓

A thin mist hangs over this eerie lakeside, where the only sound is the gentle lapping of waves. Dragon-type Pokémon rule here.



Old Gold Power Plant 🍌 🍍 🍓

Retired long ago, this vast power plant still hosts a lot of abandoned equipment — as well as many electric-type Pokémon.



Amber Canyon 🍌 🍍 🍓

This remote canyon's harsh environment has not deterred plants from adapting. Dragon- and bug-type Pokémon are abundant.

2 Choose your meal type



Curries & Stews

Simmered slow with roux, broth, and tender vegetables, these hearty bowls warm you from the inside. Rich, filling, and endlessly comforting.



Salads

Crisp vegetables and bright dressings, tossed fresh and light. From cool tofu to robust bowls, every forkful stays clean and lively.



Desserts & Drinks

Sweet treats and refreshing sips — cakes, flans, smoothies, and teas. Some lull you toward sleep, others jolt you awake.

The Experiment: v2

Accessibility is crucial for this app. Using the WCAG 2.2 AA guidelines, review the current state, identify any gaps or issues and create a plan for fixing them.

The Experiment: v2

- Native elements (<button>, <a>, <input>) used
- Icon only (no text) buttons have aria-label
- Decorative images use alt=""
- Checkboxes are label-wrapped
- **WCAG 2.2 AA gaps:** exposing state to assistive tech & document structure

The Experiment: v2

- **Island and meal pickers: no aria-checked**
- **Roster toggles: no aria-pressed**
- **Nav: no aria-current="page"**
- **"The team/meal recommendation (the app's core feature) updates silently to screen readers"**
- **Home page has no h1, no skip link on any pages**
- **No :focus-visible style, no prefers-reduced-motion, no landmarks, same title for all pages, button contrast issue**

The Experiment: v2

- **Me:** was contrast checked for text elements as well as ui ones?
- **Claude:** Fair question - the honest answer is not fully.

The Experiment: v2

- **Me:** the contrast of the text in the recommended meal card doesn't seem to be reflected in the updated plan
- **Claude:** Good catch - I never inspected RecommendedMeal's own text colours. You're right again - RecommendedMeal is full of failing text and I hadn't listed it.

The Experiment: v2

Meal



Sweet Scent Chocolate Cake

Honey ×9

Soothing Cacao ×8

Moomoo Milk ×7

Total dish size: 24 ingredients

RUNNER-UP



Craft Soda Pop

Honey ×9

Total dish size: 9 ingredients

The Experiment: v2

- axe: 0 issues
- WAVE: 9.1/10 lowest score
- Lighthouse: 100 on all pages

The Experiment: v2

Because the placeholder counts as the field's accessible *name*

axe's label rules don't check "is there a `<label>`?" — they check "does this control have a computed accessible name?" The W3C accessible-name computation has a fallback chain for text inputs:

`aria-labelledby` → `aria-label` → associated `<label>` → `title` → `placeholder`

So with no label present, the browser/axe fall back to the `placeholder` and compute the name "Search name, type, specialty or berry...". The field *does* have a name → axe's `label` rule doesn't fire → 0 violations. (My own earlier audit missed it for the same reason — it treated the placeholder as satisfying the name.)

But that's a genuine gap, not a real pass

Placeholder-as-label is a well-known anti-pattern that automated tools structurally can't catch:

- It disappears the moment you type. Once the field has content, there's no visible label — a user who looks away loses the field's purpose (relevant to 3.3.2 Labels or Instructions and general usability).
- No persistent programmatic label relationship (1.3.1 Info and Relationships) — the name only exists via the last-resort fallback, which some assistive tech treats differently (or announces as a value rather than a name).
- Low contrast — the placeholder is `night-400` (`placeholder:text-night-400`), which fails normal text contrast; placeholders get a partial exemption, but that exemption exists *because* they're not supposed to carry the label.

The Experiment: v2

- Keyboard & screen reader: better
- Uncaught issues: some

The Experiment: v3

 **Professor Potato's Sleep Selector** **Team Recommendation** Meals My Roster Clear saved data

Choose your island and the week's meal type. Professor Potato will select a team based on the selected island's berries and the strongest meal possible with matching Pokémon. Add Pokémon to your roster for results based on the Pokémon you've already studied!

1 Choose your island



Greengrass Isle 🍓 🍓 🍓

The largest island in the region, known for its fertile land covered with greenery. It hosts the greatest variety of Pokémon.



Cyan Beach 🍓 🍓 🍓

This shore of fine white sand stretches to the horizon, kissed by a gentle sea breeze. Many water-type Pokémon live here.



Taupe Hollow 🍓 🍓 🍓

Shafts of sunlight pour through the open ceiling to light up the cavern walls. Rock- and ground-type Pokémon love it here.



Snowdrop Tundra 🍓 🍓 🍓

Gorgeous flowers bloom in this freezing tundra, despite the unrelenting snowfall. Many ice-type Pokémon can be found here.



Lapis Lakeside 🍓 🍓 🍓

A thin mist hangs over this eerie lakeside, where the only sound is the gentle lapping of waves. Dragon-type Pokémon rule here.



Old Gold Power Plant 🍓 🍓 🍓

Retired long ago, this vast power plant still hosts a lot of abandoned equipment — as well as many electric-type Pokémon.



Amber Canyon 🍓 🍓 🍓

This remote canyon's harsh environment has not deterred plants from adapting. Dragon- and bug-type Pokémon are abundant.

2 Choose your meal type



Curries & Stews

Simmered slow with roux, broth, and tender vegetables, these hearty bowls warm you from the inside. Rich, filling, and endlessly comforting.



Salads

Crisp vegetables and bright dressings, tossed fresh and light. From cool tofu to robust bowls, every forkful stays clean and lively.



Desserts & Drinks

Sweet treats and refreshing sips — cakes, flans, smoothies, and teas. Some lull you toward sleep, others jolt you awake.

The Experiment: v3

- **Modern Web Guidance skill** (Google)
- **Web Design Guidelines skill** (Vercel)
- **Building Components skill** (Vercel)
- **React Best Practices skill** (Vercel)
- **Accessibility skill** (Addy Osmani)
- **React Doctor** (skill only, no CI workflow)
- **MDN MCP server** (Mozilla)

The Experiment: v3

"Using all the resources in the mcp-servers.md file, do an accessibility and best practices review of this branch only. Create a plan that explains the issues that were found and how to fix them."

The Experiment: v3

- **React Doctor scan: 32 warnings, 0 errors**
- **Full WCAG/web-interface-guidelines scan: 3 serious, 7 minor, 1 borderline**
- **"Good"/passed checks: keyboard & focus, motion/transitions, live regions/dynamic updates**

The Experiment: v3

- **Claude:** Want me to do a live keyboard/screen reader smoke test via the dev server?
- **Me:** Yes. Report on specific browser & screen reader combinations that were tested, issues found, and combinations that were excluded.

The Experiment: v3

- **Claude:** I'll be upfront about scope: this is a headless Linux container, so actual screen readers cannot run here — I can drive Chromium via Playwright for keyboard navigation and inspect the accessibility tree (what a screen reader would consume), but I can't run VoiceOver/NVDA/JAWS/Orca themselves

The Experiment: v3

Keyboard & Accessibility Smoke Test — prdc-v3

⚠ Scope & honesty note

This is a headless Linux CI container. I could drive a real browser and inspect the accessibility tree (the same role/name/state data screen readers consume), and exercise keyboard interaction — but no actual screen reader was run. Screen-reader *announcement* behavior (verbosity, reading order, phrasing) is therefore inferred from the AX tree, not observed.

Environment tested

- App under test: production build (`next build + next start`) — Next.js 16.2.10, served at `localhost:3000`.
- Driver: Playwright 1.56.1.
- Automated assertions: 21 checks — 21 passed, 0 failed.

Browser × screen-reader combinations actually tested

Browser (engine)	Version	Screen reader	Keyboard	AX-tree
Chromium (Blink), headless	141.0.7390.37	none (headless container)	✓	✓

Screen-reader combinations tested: 0. SR-facing semantics were validated through Chromium's accessibility tree only.

The Experiment: v3

Combinations excluded (and why)

- All real screen readers — VoiceOver (macOS/iOS-only), NVDA & JAWS (Windows GUI only), TalkBack (Android), Orca (needs AT-SPI + speech stack, absent here). None can run in this headless Linux container.
- Other browser engines — Firefox/Gecko and WebKit/Safari are not provisioned (only Chromium exists under `/opt/pw-browsers`), so Safari+VoiceOver and Firefox+NVDA — the two highest-value real-world pairs — were not exercised.
- Mobile (iOS Safari/VoiceOver, Chrome/TalkBack) — no device/emulator.

Recommendation

The automated keyboard + AX-tree layer is green, which is strong evidence the semantics are correct. For release confidence, a manual pass on the two canonical pairs — NVDA + Firefox (Windows) and VoiceOver + Safari (macOS) — should be done on a real OS; that's the gap this environment can't cover. I can turn `a11y-smoke.mjs` into a committed Playwright test (e.g. `@axe-core/playwright` for automated WCAG rule coverage) if you'd like it wired into the repo/CI — just say the word (and confirm adding the dev dependency, since I reverted it here).

The Experiment: v3

- **axe: 0 issues**
- **WAVE: 10/10 lowest score**
- **Lighthouse: 100 lowest score**

The Experiment: v3

Page	M3 status
Team recommendation (home)	⚠️ Partial — I fixed IslandPicker (island image → alt="") and RecommendedMeal (dish images → alt=""), but the PokemonSprite on the team tiles <i>and</i> the alternates still uses alt={pokemon.name} right next to the visible name → still redundant.
Meals (/meals)	✅ Clean — its meal/ingredient thumbnails already use alt="" (decorative); nothing to fix.
Roster (/roster)	❌ Not addressed — each roster tile renders PokemonSprite (alt={pokemon.name}) next to the visible {p.name}, so the name is announced twice (and doubles up in the toggle button's accessible name).

The Experiment: v3

Root cause: `PokemonSprite.tsx:42 (alt={pokemon.name})` is the shared culprit — it renders on the home team tiles, home alternates, and roster tiles, always adjacent to a visible name. The original review classified that alt as acceptable "informative," which is inconsistent with flagging `IslandPicker / RecommendedMeal` for the same pattern. So M3 effectively covered only the two home-page components, not the sprite that appears on home *and* roster.

Verified specifics:

- Non-empty alts remaining: `PokemonSprite (pokemon.name)` and `IslandPicker berry icons (berry name)`. The berry icons are correctly kept — the berry name isn't shown as adjacent text there, so it's genuinely informative, not redundant.
- Meals page: no non-empty alts → already clean.

The consistent fix is one change: set `PokemonSprite's alt=""` (it's always beside the visible name across all its render sites — `PokemonTile`, `TeamResult alternates`, `roster`). That closes the gap on both the home and roster pages at once.

The Experiment: v3

- Keyboard & screen reader: **focus management + disabled controls**
- Uncaught issues: **none**

Aside: WebAIM 1 Million

<https://webaim.org/projects/million/>

- Detected errors **increased 10.1%** since 2025
- Average of **56.1 errors** per page
- Most common errors: **low contrast text**, **missing alt text**, **missing labels**, **empty links/buttons**, **missing language**

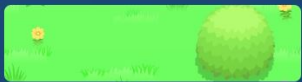
The Experiment: v4

 **Professor Potato's Sleep Selector** [Clear saved data](#) [Team Recommendation](#) [Meals](#) [My Roster](#)

Team Recommendation

Choose your island and the week's meal type. Professor Potato will select a team based on the selected island's berries and the strongest meal possible with matching Pokémon. Add Pokémon to your roster for results based on the Pokémon you've already studied!

1 Select your island



Greengrass Isle ✓ 🍌 🍍 🍓 🍇
The largest island in the region, known for its fertile land covered with greenery. It hosts the greatest variety of Pokémon.



Cyan Beach 🌊 🐚 🦀 🍹
This shore of fine white sand stretches to the horizon, kissed by a gentle sea breeze. Many water-type Pokémon live here.



Taupe Hollow 🍄 🍌 🍍
Shafts of sunlight pour through the open ceiling to light up the cavern walls. Rock- and ground-type Pokémon love it here.



Snowdrop Tundra ❄️ 🌸 🍌 🍍
Gorgeous flowers bloom in this freezing tundra, despite the unrelenting snowfall. Many ice-type Pokémon can be found here.



Lapis Lakeside 🌊 🐉 🍌 🍍
A thin mist hangs over this eerie lakeside, where the only sound is the gentle lapping of waves. Dragon-type Pokémon rule here.



Old Gold Power Plant ⚡ 🍌 🍍
Retired long ago, this vast power plant still hosts a lot of abandoned equipment — as well as many electric-type Pokémon.



Amber Canyon 🏜️ 🐉 🍌 🍍
This remote canyon's harsh environment has not deterred plants from adapting. Dragon- and bug-type Pokémon are abundant.

2 Select your meal type



Curries & Stews ✓
Simmered slow with roux, broth, and tender vegetables, these hearty bowls warm you from the inside. Rich, filling, and soul-soothing.



Salads
Crisp vegetables and bright dressings, tossed fresh and light. From cool tofu to robust bowls, every forkful stays pleasant.

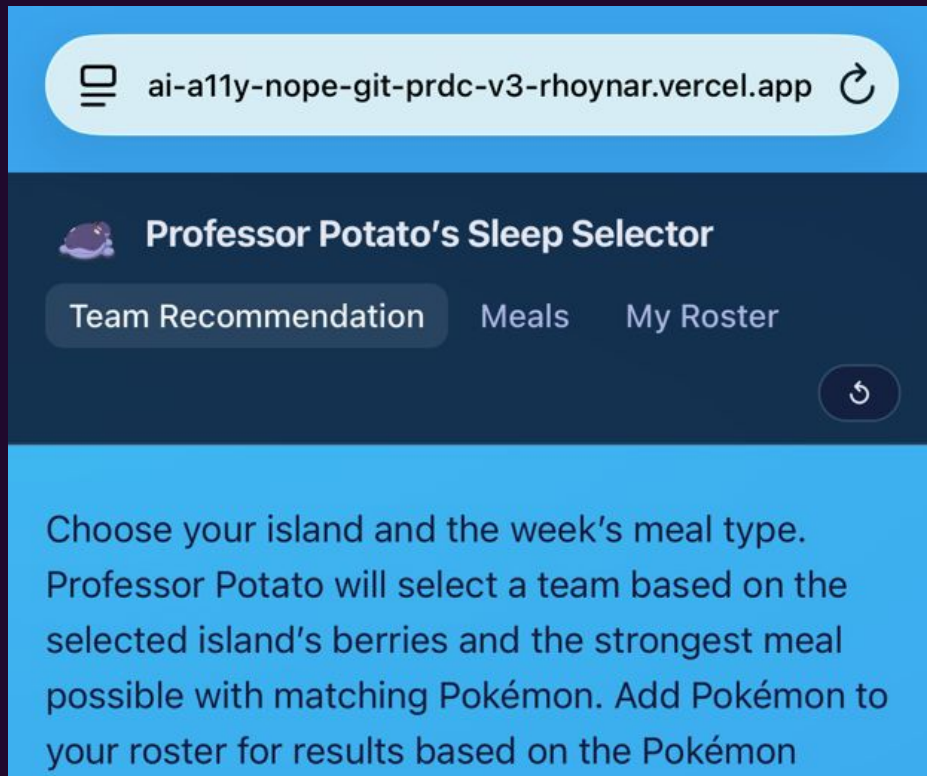


Desserts & Drinks
Sweet treats and refreshing sips — cakes, flans, smoothies, and teas. Some lull you toward sleep, others with energizing.

bit.ly/ai-a11y-nope

@ashleiglodge.ca

The Experiment: v4



The Experiment: v4

You're absolutely right — that's a WCAG 2.4.3 (Focus Order) / 1.3.2 (Meaningful Sequence) violation. I used CSS `order-last`, which reorders *visually* but not in the DOM, so keyboard/screen-reader order stayed logo → nav → reset while the screen showed logo → reset → nav. CSS `order` must never create that mismatch.

The correct fix is to make the DOM order match the visual order instead of using `order`. I'll put the reset button after the logo in the DOM (logo → reset → nav) so tab order and visual order agree at every breakpoint:

Read and edited Nav.tsx +13 -11 >

Now let me add a focus-order guard that directly catches this class of bug — tab order must equal the visual reading order (row-major). This would have failed on the previous `order`-based version:

Edited a11y-smoke.spec.ts, ran 3 commands +25 -0 >

The Experiment: v4

No — it only runs on the home page, and it only checks the header. Specifically:

- **Pages:** just / (line 266). It does *not* run on /meals or /roster. Only the "has no horizontal overflow" test loops over all three pages; the focus-order test doesn't.
- **Scope:** only header controls — the `if (!a || !a.closest('header'))` filter (line 273) discards anything outside `<header>`. So it verifies the nav's order, not the page-body tab order.
- **Viewport:** only 320px (it's inside the Mobile layout (320px) describe). It doesn't exercise the desktop single-row layout, whose order my fix also changed.

The practical coverage is decent because the `<Nav>` is one shared component rendered identically on every page — so the home run is representative of the nav everywhere. But strictly, the guard is home-only, header-only, mobile-only.

If you'd like tighter coverage, I can extend it (cheap, all in the test file on `prdc-v4`):

1. Run the header check on all three pages — guards against a page-specific stacking/CSS quirk affecting the header.
2. Add a desktop-width variant — my fix reordered the desktop layout too (reset moved next to the brand), so this would lock that order in.
3. Optionally, check body focus order — more involved and higher-maintenance; I'd generally skip it unless you want it.

The Experiment: v4

Option 1: Disabled and not clickable, but can be "activated" by AT and seem broken

Option 2: Enabled and clickable, no visual change, but also no info for AT on why the button is "broken"

The Experiment: v4

Given the following requirements for accessibility & general usability, suggest 1-3 optimal solutions for handling disabled buttons. Include information on how each solution meets or doesn't meet the requirements. Check/use the skills available in the repo, the WCAG 2.2 AA guidelines, and other available knowledge sources for information on overall best practices & solutions.

1. If the button is disabled, it must be clearly apparent visually and to assistive technology such as screen readers that the button is disabled.
2. If the button is disabled, information must be provided to the user (visually and to assistive technologies) what they must do to enable the button/actually perform the action.
3. If the button is not disabled, information must be provided to the user as to why the expected action (ie. clearing the search, increasing the pot size) isn't happening, both visually and for assistive technologies.
4. The user cannot be trapped/stuck on the button (ie. unable to move forward via keyboard or screen reader navigation), and focus should not move unexpectedly when the button is either focused or when it loses focus.

The Experiment: v4

First: the anti-pattern to rule out — native disabled

The HTML disabled attribute (`<button disabled>`) fails your requirements and is exactly what caused the bugs you hit earlier on the pot steppers and Clear button:

- Fails Req 2 — a natively-disabled control is removed from the tab order, so a keyboard/screen-reader user *tabbing through the page never lands on it* and never hears any "here's how to enable it" hint. (`aria-describedby` is also announced unreliably on disabled controls.)
- Fails Req 4 — a natively-disabled element cannot hold focus. If it becomes disabled *while focused* (e.g. you click "+" until the pot hits max), the browser drops focus to `<body>` → focus moves unexpectedly. This is the "focus jumps somewhere else" bug from before.
- It also caused the iOS VoiceOver double-tap-still-activates oddity.

So every viable solution below uses `aria-disabled="true"` or no disabling at all — never the native attribute.

The Experiment: v4

Solution 1 — Accessible-disabled (aria-disabled + aria-describedby)

Keep the button a real, focusable `<button>` (never disabled); mark state with `aria-disabled="true"`; guard the handler to no-op; and point `aria-describedby` at a visible reason/instruction.

```
<button
  type="button"
  aria-disabled={atMax}
  aria-describedby="pot-bound"
  onClick={() => { if (atMax) return; /* no-op when blocked */ setPot(p => p + 1); }}
  className={atMax ? 'opacity-50' : ''} // focus ring stays full-strength
>+</button>

{atMax && <span id="pot-bound" className="text-xs text-night-100">Pot is at its maximum
(15)</span>}
```

The Experiment: v4

Solution 2 — Always-enabled + live-region feedback (enabled-but-inert)

Never disable. The button is always focusable and activatable; when the action can't take effect it clamps or no-ops, and an `aria-live` region explains the result. This is what the repo currently does for Clear selection ("Selection cleared.") and the pot steppers ("Maximum size" / "Minimum size" badge).

```
<button type="button" onClick={onClear} className="btn-ghost">Clear selection</button>
<p aria-live="polite" className="sr-only">{statusMsg}</p>
```

The Experiment: v4

Solution 3 — Hybrid (recommended for a mixed UI)

Choose per control based on whether the block is a user-actionable precondition or a boundary/no-op:

- `aria-disabled` + `aria-describedby` (Sol. 1) when the user must *do something specific* to unlock it, and the instruction is worth surfacing on focus.
- Always-enabled + live region (Sol. 2) for boundary conditions and idempotent actions where explaining the result on activation is clearer than a permanently dimmed control.

This is the only approach that satisfies all four requirements across a UI, because Reqs 1-2 (disabled semantics) and Req 3 (enabled semantics) are mutually exclusive *per control* — you can't be both disabled and enabled at once. Req 4 holds under both because neither uses native `disabled`.

Cost: two patterns to maintain and a judgment call per control.

The Experiment: v4

Recommendation for this app

Your controls are boundary/idempotent (pot at min/max, clear an already-empty selection), so **Solution 2** is the right default and you already ship it correctly — always-enabled, clamp/no-op, visible "Maximum size"/"Minimum size" badge with `aria-describedby`, `aria-live` announcements, and full-strength focus rings.

I'd only reach for **Solution 1** if you add a control gated on a real user precondition (e.g. a future "Save team" that needs 5 Pokémon selected) — there, `aria-disabled` + a focusable "Select 5 Pokémon to save" hint directly satisfies Reqs 1-2 without the native-disabled focus/tab pitfalls.

The one rule that's non-negotiable across all of them: **never the native disabled attribute**, because it alone fails Req 2 and Req 4.

Relevant WCAG 2.2 AA criteria: 4.1.2 (Name/Role/Value), 1.4.1 (Use of Color), 3.3.1/3.3.3 (Error identification/suggestion), 4.1.3 (Status messages), 2.1.2 (No keyboard trap), 2.4.3 (Focus order), 2.4.7/2.4.11 (Focus visible / not obscured).

The Experiment: v4

- **axe: 0 issues**
- **WAVE: 10/10 lowest score**
- **Lighthouse: 100 lowest score**
- **Keyboard & screen reader: none**
- **Uncaught issues: ?? (you tell me!)**

**Accessibility is
about people, not
machines**

**AI for accessibility?
Maybe!**

Resources

- AIMAC: The AI Model Accessibility Checker **benchmark analysis**
- Can Your AI Pass the Accessibility Test? **presentation**
- The Web Is Being Made Accessible for AI, Not People **article**
- **The Impact of AI on Digital Accessibility,** Part I & Part II **articles**

Resources

- OpenAI, ARIA, and SEO: Making the Web Worse **article**
- AI and Accessibility: Promise, Limitations, and the Importance of Human Expertise **article**
- Accessibility is resistance! **article**
- AI Acceleration Whiplash **report takeaways**
 - **Consider #3 and #4 re: digital accessibility**

Resources

- MDN MCP Server announcement
- Modern Web Guidance skill from Google
- skills.sh skills directory from Vercel
- Accessibility skill by Addy Osmani
- Web Design Guidelines skill from Vercel

Resources

- Building Components skill from Vercel
- React Best Practices skill from Vercel
- Accessibility Review skill from Anthropic (note: WCAG 2.1 only)
- React Doctor deterministic code scanner
- CodeRabbit AI code reviews

THANKS!



Nation



AMA!



Ashleigh Lodge

Neovation Learning Solutions

[@ashleighlodge.ca](https://www.ashleighlodge.ca)

[Linktree](#)

[ashleighlodge.ca](https://www.ashleighlodge.ca)